

been passed an erroneous input, and (b) takes steps to deal with it appropriately. To ensure that, you need to have tests that can cover the input space of your application.

Given that input spaces can be quite huge, you cannot have tests that check each single point in your application's input space. The idea is to cover the application input space with a set of tests that partition your input space so that they cover all of it. Since 100 per cent coverage may be expensive, both computationally and with regard to time, one may opt to choose a subset of tests which cover as much as possible within a given budget of time and computational resources. One also wants to make sure that the application deals gracefully with all the possible error paths that can arise during execution. Hence, it is important to design tests that cover the application code exhaustively. An alternate way is thinking of the path coverage/decision coverage of your application, where you want to make sure that all possible values for an outcome are tested, all possible combinations of condition statements are tested, and each entry and exit point is covered.

You may have been wondering why we were discussing testing standard software so much, when our focus should be on testing NLP applications. There are well known methodologies, tools and quality assurance (QA) solutions for standard software testing. I would like our readers to pause in their reading and think about two questions:

1. How much of standard software testing methodology is applicable to testing NLP applications?
2. What are the tools, techniques and QA practices that are available for testing NLP applications?

At first glance, it appears as though existing software testing methodologies are directly applicable to NLP testing also. Let us assume that we are writing our NLP application in Python. There are well known test frameworks in Python, and we can do unit testing, regression testing, input space coverage testing and code coverage using them. Well, now ask yourself the next question — is that sufficient to test your NLP application? Unfortunately, the answer is that it is not. Most machine learning or NLP applications are probabilistic in nature (in the sense that even if they return categorical labels, the labels are associated with a probabilistic score and are not absolute). Hence, there is no test oracle that can enumerate the correct output value for every possible input. Without a test oracle, how would you test all possible (input, output) combinations?

Also note that since ML/NLP systems are by nature predictive and probabilistic, we do not expect that they will get all (input, output) combinations correct. Instead, there are acceptable criteria of correctness. For instance, let us assume that you are building an application that can detect the sentiment associated with individual tweets. You can specify that you will be able to achieve a precision of above 80 per cent and recall of above 90 per cent on your test data set.

Instead of a test oracle, you can use your pre-labelled test data ground truth to verify that your application does indeed meet the criteria you have agreed on beforehand.

Now think about input space coverage testing. Given the high dimensional input spaces, how can you ensure that you will be able to cover the input space in your testing? More importantly, without a test oracle, how will you know the correct output value for each of your zillion possible inputs? You only have limited labelled test data for which you have the ground truth.

Now think about code coverage. Is it sufficient to achieve 100 per cent code coverage of your NLP application? What will happen if you feed it inputs that are from a distribution that is different from the one it has been trained on? Can your application first recognise that its inputs are from a different distribution, and next, can it gracefully handle this difference?

Let us think about one more challenge. Assume that your NLP application uses a cloud service to obtain the sentiment associated with a piece of text. There are a number of cloud based NLP tools such as Google Cloud NLP or Microsoft Cognitive Services Toolkit, where you can submit a piece of text and obtain the sentiment associated with it. Now let us assume that the cloud based service supports active learning, i.e., that if it labels tweets wrongly, users can teach it the correct labels. Now you have a component of an application, the behaviour of which can be modified by the number of testing examples you provide to it.

Is it possible for a malicious user to provide incorrect examples to mislead the component into making wrong decisions since it learns the wrongly labelled inputs? Given all these challenges and questions, how do we go about testing NLP applications?

We need to focus on three major components as part of our testing — data, features and models/algorithms. For each of these, I request our readers to think about the testing they would design. For instance, think of a real/imaginary NLP application you have created or would like to create. Now write down how you would test (a) the input data, (b) the features, and (c) the models. Next, think about what tools/techniques you will use for each of these steps. We will discuss this in detail in next month's column.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Wishing all our readers happy coding until next month. **END** 🐧

By: Sandya Mannarswamy

The author is an expert in systems software and is currently working as a research scientist in Conduent Labs India (formerly Xerox India Research Centre). Her interests include compilers, programming languages, file systems and natural language processing.